

NanoFlow: Towards Optimal Large Language Model Serving Throughput

KAN ZHU, University of Washington, USA

YUFEI GAO, University of Washington Tsinghua University, USA

YILONG ZHAO, University of Washington UC Berkeley, USA

LIANGYU ZHAO, University of Washington, USA

GEFEI ZUO, University of Michigan, USA

YILE GU, University of Washington, USA

DEDONG XIE, University of Washington, USA

TIAN TANG, University of Washington Tsinghua University, USA

QINYU XU, University of Washington Tsinghua University, USA

ZIHAO YE, University of Washington, USA

KEISUKE KAMAHORI, University of Washington, USA

CHIEN-YU LIN, University of Washington, USA

ZIREN WANG, University of Washington Tsinghua University, USA

STEPHANIE WANG, University of Washington, USA

ARVIND KRISHNAMURTHY, University of Washington, USA

BARIS KASIKCI, University of Washington, USA

Large Language Models (LLMs) have resulted in a surging demand for planet-scale serving systems, where tens of thousands of GPUs continuously serve hundreds of millions of users. Consequently, throughput has emerged as a key metric that determines serving systems’ performance. Due to large model sizes and memory-intensive self-attention, LLM serving has been commonly assumed to be memory-bound. Through a detailed analysis, we show that despite having memory-intensive components, end-to-end LLM serving is compute bound for most common workloads and LLMs. Alas, most existing serving engines fall short from optimal compute utilization, because the heterogeneous operations that comprise LLM serving—compute, memory, networking—are executed sequentially within a device.

We propose NanoFlow, a novel serving framework that exploits intra-device parallelism, which overlaps the usage of heterogeneous resources within a single device. NanoFlow splits inputs into smaller nano-batches and duplicates operations to operate on each portion independently, enabling overlapping. NanoFlow automatically identifies the number, size, ordering, and GPU resource allocation of nano-batches to minimize the execution time, while considering the interference of concurrent operations. We evaluate NanoFlow’s end-to-end serving throughput on several popular models such as LLaMA-2-70B, Mixtral 8×7B, LLaMA-3-8B, etc. With practical workloads, NanoFlow provides 1.91× throughput boost compared to state-of-the-art serving systems, achieving between 50% to 72% of optimal throughput across popular models.

Additional Key Words and Phrases: Large Language Models, Inference Serving, Throughput, GPU Utilization, Intra-device Parallelism, Nano-batching, Pipeline Scheduling

Authors’ Contact Information: Kan Zhu, University of Washington, USA; Yufei Gao, University of Washington and Tsinghua University, USA; Yilong Zhao, University of Washington and UC Berkeley, USA; Liangyu Zhao, University of Washington, USA; Gefei Zuo, University of Michigan, USA; Yile Gu, University of Washington, USA; Dedong Xie, University of Washington, USA; Tian Tang, University of Washington and Tsinghua University, USA; Qinyu Xu, University of Washington and Tsinghua University, USA; Zihao Ye, University of Washington, USA; Keisuke Kamahori, University of Washington, USA; Chien-Yu Lin, University of Washington, USA; Ziren Wang, University of Washington and Tsinghua University, USA; Stephanie Wang, University of Washington, USA; Arvind Krishnamurthy, University of Washington, USA; Baris Kasicki, University of Washington, USA.

1 Introduction

Transformer-based large language models now sit behind chatbots, search, and office autocomplete [20, 27, 39], with weekly ChatGPT usage reportedly in the hundreds of millions [30, 34, 38]. Every interaction consumes scarce, expensive GPU time [10, 45], so operators optimize for a single objective—tokens per second per device. Throughput has become the dominant cost metric [12, 16, 25].

Serving an LLM is structurally harder than serving earlier networks. Model weights are enormous—GPT-3’s 175B parameters need roughly five 80 GB A100s just for FP16 weights [7]—and self-attention [49] adds per-request state that grows with context length. Production systems cache per-request keys and values in a *KV-cache* [33] to avoid recomputation, but for long-context workloads that cache can exceed the weights themselves [42]. Because a decode step reads all weights plus the entire KV-cache to emit one token per sequence, its arithmetic intensity looks terrible in isolation.

The field’s natural conclusion is that serving is memory-bandwidth bound. The paper we reproduce argues this is wrong at the granularity that matters, because it ignores *batching*. Once hundreds of requests are batched, weight-loading cost amortizes across the batch and the dense matrix multiplications that dominate a layer become arithmetic-limited rather than bandwidth-limited. Grouped-query attention [4] further shrinks per-request KV state so batches grow larger still, and as parameter counts climb the GEMM work grows faster than attention work. Evaluated end-to-end rather than operation-by-operation, mainstream serving sits firmly in a *compute-bound* regime.

The corollary is uncomfortable: if compute binds, the right efficiency metric is achieved FLOP/s against hardware peak, and today’s engines score poorly. Each kernel individually runs near 80% of *its own* bottleneck resource—a GEMM saturates tensor cores, decode attention saturates memory, an AllReduce saturates NVLink—but because they are dispatched sequentially onto one GPU, whichever resource limits throughput sits idle whenever another kernel runs. Aggregate compute utilization across a layer collapses to roughly 40%.

NanoFlow attacks this waste with intra-device parallelism: it runs operations of *different* resource classes concurrently on one device, so a compute-bound GEMM and a memory-bound attention kernel occupy disjoint hardware at the same instant. To break the data dependency that serializes them, NanoFlow splits each batch into *nano-batches* and replicates each operation into *nano-operations*, one per nano-batch. Because nano-batches hold disjoint requests, their nano-operations are independent and freely interleavable. Splitting into k nano-batches reloads weights k times, raising memory traffic; the arbitrage is that a compute-bound workload has idle memory bandwidth, so the extra loads hide in memory-idle windows at no wall-clock cost. NanoFlow further hands each nano-operation only a *fraction* of the GPU’s SMs, since bandwidth-bound kernels saturate with far fewer thread blocks than compute-bound ones.

Building this raises two problems. *The scheduling search space is enormous*: the number, size, and ordering of nano-operations depend on hidden dimension, layer count, GQA/MoE structure, the GPU’s compute-to-bandwidth ratio, and the prefill/decode mix—hand-tuning per (model, hardware, workload) does not scale. *Concurrent kernels interfere unpredictably*: two kernels sharing an SM contend for warp slots, L2, and memory-controller queues [41], and the slowdown is non-linear and asymmetric. NVIDIA GPUs expose no interface to partition compute, bandwidth, and network directly, so the trade-off must be inferred empirically.

NanoFlow resolves both with *auto-search*, a two-stage MILP optimizer. Stage I ignores interference and solves the pipeline structure—counts, sizes, and ordering of nano-operations—keeping the combinatorial part tractable. Stage II fixes that structure and allocates each concurrent nano-operation a GPU fraction using an empirically profiled interference model. A runtime forms batches asynchronously, enforces the ordering with CUDA events, and offloads KV-cache

to a CPU/SSD hierarchy. On LLaMA-2-70B across eight A100s, NanoFlow reaches 68.5% of the cost-model optimum, averages a $1.91\times$ throughput gain over TensorRT-LLM on realistic traces (ShareGPT, LMSYS-Chat-1M, Splitwise), and sustains $1.64\times$ the request rate of the best baseline within a 200 ms latency SLO. Auto-search generalizes without manual tuning to LLaMA-3-8B/70B, Qwen2-72B, Deepseek-67B, and Mixtral 8×7 B, reaching 50–72% of optimal on each.

The original work’s contributions, restated:

- An analytical cost model of LLM serving—validated against kernel measurements—that establishes modern workloads as compute-bound and yields a closed-form optimal-throughput expression.
- NanoFlow: an MILP auto-search engine that discovers intra-device pipelines automatically, plus an end-to-end runtime that executes them.
- An evaluation showing $1.91\times$ throughput over state-of-the-art baselines, at 68.5% of the derived ceiling, across six architectures.

Section 2 covers the inference workflow and operation classification; Section 3 presents the cost model and NanoFlow’s design; Section 4 reports results; and Section 5 offers a critical assessment.

2 Background and Related Work

2.1 The LLM Inference Workflow

Contemporary production models—GPT-4, LLaMA, Mistral [21, 28, 47]—are decoder-only transformers [49] served in two phases [51]. *Prefill* ingests the whole prompt at once as a single large matrix, writing keys and values into the KV-cache. *Decode* then generates one token at a time, each step conditioned on all prior tokens, with the KV-cache sparing decode step t from recomputing keys and values for tokens $1 \dots t-1$ [33].

Both phases traverse the same stack of identical decoder layers (Figure 1). Within a layer, the input activation is projected by W_Q , W_K , W_V into queries, keys, and values; the new keys/values are appended to the KV-cache. The query is scored against every cached key, normalized into attention weights, and used to form a convex combination of cached values, which passes through the output projection W_O and a layer norm [6]. The feed-forward block then projects the activation through W_{up} and W_{gate} , applies a nonlinearity such as SiLU [9] to the gate and multiplies elementwise, and projects back through W_{down} to the model dimension for the next layer.

2.2 Classifying Operations by Bottleneck Resource

The paper’s central move is to sort a layer’s operations by *which hardware resource each saturates*. Four classes emerge.

Dense operations. KQV generation, the O-projection, and the Up/Gate/Down projections multiply activations against weight matrices. Prefill and decode use the same weights, so they fuse into one GEMM—chunked prefill [3]—amortizing weight loading across a large batch. Dense operations are unambiguously *compute-bound*.

Attention operations. Prefill attention scores a whole prompt against itself—a matrix-matrix product, *compute-bound*. Decode attention scores a single query against a request-unique KV-cache—a matrix-vector product that reads a large tensor for few FLOPs, *memory-bound*.

Network operations. Under tensor parallelism, per-layer partial results are reconciled across GPUs by AllGather (AG) and AllReduce (AR) collectives, limited by interconnect bandwidth (NVLink [24]): *network-bound*.

Other operations. Layer norms, positional embeddings, and activations—individually cheap, folded into a residual category.

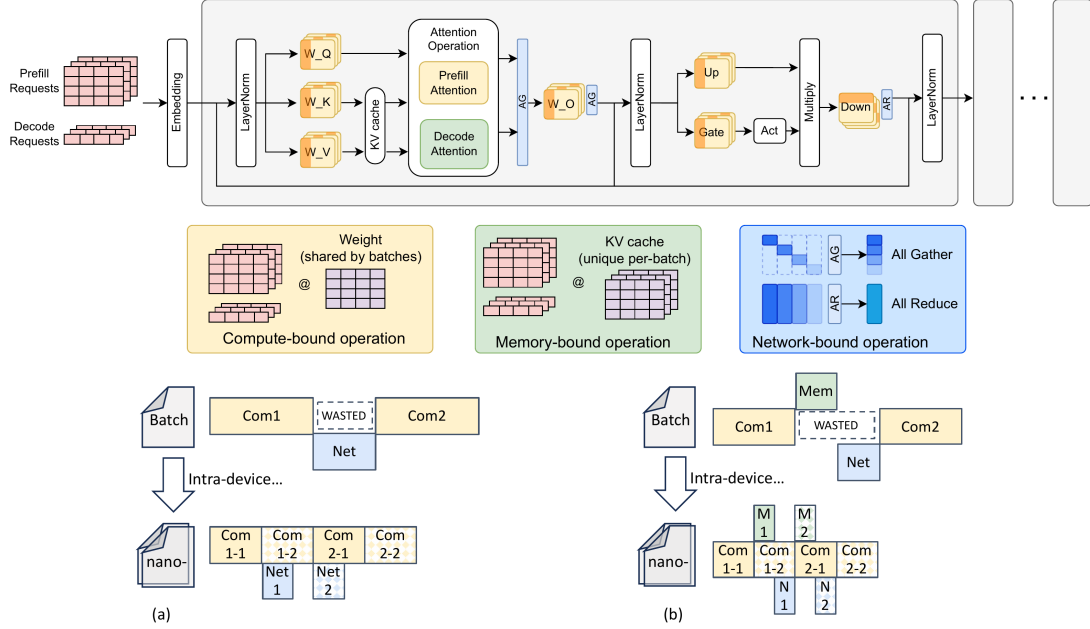


Fig. 1. Transformer architecture, reproduced from the original paper. The operations in the yellow boxes have large batch sizes and share model weight parameters across requests; hence, they are compute-bound. Operations in green boxes require loading a unique KV cache for each request; hence, they are memory-bound. The blue box represents network operations that perform synchronization between operations.

The heterogeneity is the point: a GEMM and a decode-attention kernel stress disjoint hardware, so running them sequentially idles one resource while the other works. Running them concurrently—absent a data dependency—keeps both busy, which is exactly what NanoFlow arranges.

2.3 Multi-GPU Serving

Large models do not fit on one device [17], so systems partition them [11, 36, 55]. **Tensor parallelism** [36, 55] shards each weight matrix across GPUs; every device participates in every operation, so compute and memory scale with device count, at the cost of a collective after nearly every operation (the AG/AR calls in Figure 1). **Pipeline parallelism** [13] instead splits the model by depth, minimizing communication but introducing pipeline bubbles. In practice they compose—tensor parallelism within a node, pipeline across nodes—and this evaluation uses TP=8 within one DGX node.

2.4 Related Work

Serving optimizations at coarser granularities. Prior work attacks serving efficiency above the individual operation. Orca [51] introduced continuous batching at iteration granularity; DistServe [57] and Splitwise [31] disaggregate prefill and decode onto separate clusters; DeepSpeed-FastGen [12] and Sarathi-Serve [2] chunk prefills and co-batch them with decodes. All schedule *requests*, none schedules *operations*, so none can co-locate a GEMM and a decode-attention on the GPU—the gap NanoFlow targets. A parallel thread improves individual kernels (PagedAttention [16], quantization [18, 42, 53]); NanoFlow is complementary, since a better kernel is simply re-profiled into auto-search.

Operator-level parallelism in DNN compilers. Rammer [19], Unity [48], ASPEN [29], and Welder [35] expose intra-operator parallelism or dissolve operator boundaries into tile-level dataflow, but optimize for parallelism without modeling that different operators are limited by *different physical resources*; their tiles inherit the graph’s sequential dependencies, capping expressible overlap, and they generally demand per-model recompilation. NanoFlow’s nano-batching sidesteps the dependency problem by construction, and auto-search removes the manual effort.

3 Method

This section develops the cost model that identifies the bottleneck and validates it (§3.1–§3.3), the optimal-throughput bound and the gap real systems leave (§3.4–§3.5), then NanoFlow’s mechanism (§3.6), auto-search (§3.7), and runtime (§3.8).

3.1 A Cost Model for LLM Serving

3.1.1 Parameters. Throughput means *total* tokens per second, prefill and decode alike, governed by four parameter groups.

Hardware. N_{GPU} ; MemBW (aggregate bandwidth); MemSize (capacity); Compute (aggregate FLOP/s); NetBW (interconnect bandwidth).

Model. D_{model} (hidden dimension); L (layers); P_{Model} (parameters); R_{GQA} ; S_{type} (bytes/parameter, 2 for FP16).

Workload. p (mean prompt tokens), d (mean generated tokens); a request contributes $p + d$ tokens, so decode throughput is $d/(p + d)$ Throughput_{total} and request rate is $1/(p + d)$ Throughput_{total}.

Batch size. The model assumes the largest batch memory permits—weights plus every active request’s KV-cache—which is right for a throughput-oriented system: larger batches amortize weight loading and dilute fixed launch costs. Activations are neglected (<5% of memory [16]).

3.1.2 Latency from Three Perspectives. Under the max-batch assumption, one iteration (a full forward pass through all L layers) can be costed three ways.

Memory. At max batch, essentially all device memory is touched once per iteration—weights streamed in, every KV-cache read by decode attention—and cross-iteration weight caching is hopeless because the reuse distance spans the whole forward pass:

$$T_{\text{mem}} = \frac{\text{MemSize}}{\text{MemBW}}. \quad (1)$$

Compute. Dense operations dominate FLOPs (validated in §3.3). A GEMM against a weight matrix $N_w \times K_w$ at batch B_{Dense} costs $2B_{\text{Dense}}N_wK_w$ FLOPs; summing over all weight matrices gives $2B_{\text{Dense}} \sum N_wK_w$, and $\sum N_wK_w = P_{\text{Model}}$. Therefore

$$T_{\text{compute}} \approx \frac{2B_{\text{Dense}} \cdot P_{\text{Model}}}{\text{Compute}}. \quad (2)$$

Crucially, compute time depends on the parameter count and nothing else about the architecture.

Network. Tensor parallelism moves, per layer, two AllGathers and one AllReduce [37] over $[B_{\text{Dense}}, D_{\text{model}}]$ activations, totaling $4 B_{\text{Dense}}D_{\text{model}}S_{\text{type}}$ per-GPU bytes per layer:

$$T_{\text{net}} \approx 4 \cdot \frac{N_{\text{GPU}} B_{\text{Dense}} D_{\text{model}} S_{\text{type}} L}{\text{NetBW}}. \quad (3)$$

Table 1. Characteristics of accelerator models across various vendors and release years. Recreated from Table 1 of the original paper. The three rightmost columns are the ratios that matter: they are conspicuously flat across a decade of hardware, which is why the compute-bound conclusion is not A100-specific.

Vendor	Model	Release Year	MemSize (GB)	MemBW (GB/s)	NetBW (GB/s)	Compute (FP16 GFLOP/s)	MemSize MemBW	Compute MemBW	NetBW MemBW
NVIDIA	V100	2017	16	900	300	125,000	0.018	139	0.33
NVIDIA	A100 (40GB)	2020	40	1,555	600	312,000	0.026	200	0.39
NVIDIA	A100 (80GB)	2021	80	2,000	600	312,000	0.040	156	0.30
NVIDIA	H100	2023	80	3,352	900	989,000	0.024	295	0.268
NVIDIA	H200	2024	96	4,800	900	989,000	0.020	206	0.19
NVIDIA	B100	2024	120	8,000	1,800	1,800,000	0.015	225	0.23
NVIDIA	B200	2024	120	8,000	1,800	2,250,000	0.015	281	0.23
AMD	MI250	2021	128	3,352	800	362,000	0.038	107	0.24
AMD	MI300	2023	192	5,300	1,024	1,307,000	0.036	246	0.19
AMD	MI325X	2024	256	6,000	1,024	1,307,000	0.043	218	0.17
Intel	Gaudi 2	2022	96	2,400	600	1,000,000	0.040	417	0.25
Intel	Gaudi 3	2024	128	3,700	1,200	1,800,000	0.035	486	0.32
NVIDIA	Ada 6000	2022	48	960	64	182,000	0.050	190	0.067

3.2 Which Resource Actually Binds?

3.2.1 *Network versus Compute.* Dividing Eq. 3 by Eq. 2:

$$\frac{T_{\text{net}}}{T_{\text{compute}}} = \frac{2D_{\text{model}}L}{P_{\text{Model}}} \cdot \frac{N_{\text{GPU}} \cdot \text{Compute}}{\text{NetBW}/S_{\text{type}}}. \quad (4)$$

Standard parameter accounting (with intermediate dimension $3.5 \times D_{\text{model}}$ [15, 21]) gives $P_{\text{Model}} \approx 12D_{\text{model}}^2 L$, collapsing the first factor to $\approx 1/(6D_{\text{model}})$. Any model needing multiple GPUs has $D_{\text{model}} > 4096$ [5, 15, 21, 46], so this factor is below 10^{-5} , while modern interconnects (NVLink [24], Infinity Fabric [52]) put the second factor at 10^4 – 10^5 . The product lands below 1: *the network is not the bottleneck.*

3.2.2 *Memory versus Compute.* The decisive comparison. Dividing Eq. 1 by Eq. 2:

$$T_R = \frac{T_{\text{mem}}}{T_{\text{compute}}} \approx \frac{\text{Compute}}{\text{MemBW}} \cdot \frac{\text{MemSize}}{P_{\text{Model}}} \cdot \frac{1}{2B_{\text{Dense}}}. \quad (5)$$

$T_R < 1$ means compute-bound.

Two trends push T_R down. GQA [4]—now standard in LLaMA-3 [46], NeMo [23], and Qwen2 [5]—shrinks per-request KV state so far more requests fit, raising B_{Dense} ; the authors report LLaMA-2-70B on 8×A100 sustaining $B_{\text{Dense}} \approx 2048$ where a non-GQA model would manage 256, an 8× swing. *Model growth* only increases P_{Model} in the denominator. Across five models and six workloads nearly every T_R is below 1 (compute-bound); the lone exception is LLaMA-3-8B under a long-decode (512-in/1024-out) workload, where $T_R \approx 1$. Table 1 reproduces the accelerator survey: the ratios that matter (Compute/MemBW, NetBW/MemBW) are remarkably stable across vendors and eight years of silicon, so the conclusion is not A100-specific.

3.3 Validating the Model

The authors validate the model on LLaMA-2-70B, 8×A100, $B_{\text{Dense}} = 2048$: per-operation FLOPs, bytes, and network traffic are fed through Eqs. 1–3 and compared to measured kernel times (Table 2). Predictions track measurement

Table 2. Comparison of operation runtimes between cost model estimation and real-world measurements. Recreated from Table 2 of the original paper. LLaMA-2-70B, $8 \times A100$, $B_{\text{Dense}} = 2048$. UG = Up+Gate; D = Down; DecAttn = decode attention; PfAttn = prefill attention; $T_{\text{comp}}/T_{\text{mem}}/T_{\text{net}}$ = estimated compute/memory/network time.

Op	Compute (GFLOP)	Mem Load (GB)	Net Usage (GB)	Est. T_{comp} (ms)	Est. T_{mem} (ms)	Est. T_{net} (ms)	Real Time (ms)
KQV	27,487.8	19.5	0	11.01	1.22	0	16.08
O	21,990.2	16.1	0	8.81	1.01	0	16.01
UG	153,931.6	96.6	0	61.67	6.04	0	69.92
D	76,965.8	49.7	0	30.84	3.11	0	34.96
DecAttn	3,665.9	462.2	0	1.47	28.89	0	35.60
PfAttn	916.3	2.1	0	0.37	0.13	0	4.56
Net	18.8	75.2	75.2	0.01	4.70	31.33	47.92
Total				114.17	45.09	31.33	

well; the one clear miss is prefill attention (0.37 ms predicted vs. 4.56 ms measured), which the authors attribute to kernel-launch overhead on a too-small kernel—an honest disclosure that does not move the aggregate. Summing the columns, $T_{\text{compute}} = 114.17$ ms exceeds $T_{\text{mem}} = 45.09$ ms by $2.5\times$ and $T_{\text{net}} = 31.33$ ms by $3.6\times$. Notably, 462 GB of the memory load is decode attention alone—the very operation cited to call serving memory-bound. The operation is memory-bound; the layer is not.

3.4 The Optimal Throughput Bound

If compute is the binding constraint, peak throughput is achieved exactly when the tensor cores never idle. Rearranging Eq. 2:

$$\text{Throughput}_{\text{optimal}} = \frac{B_{\text{Dense}}}{T_{\text{compute}}} = \frac{\text{Compute}}{2P_{\text{Model}}}. \quad (6)$$

Two quantities set the ceiling—aggregate FLOP/s and parameter count. Memory capacity, bandwidth, prompt length, and generation length all drop out; they determine only how close an implementation gets, not where the ceiling lies. For LLaMA-2-70B on $8 \times A100$, profiling CUTLASS [43] gives a realistic 280 TFLOPS FP16 peak, so with $P_{\text{Model}} = 70\text{B}$ the ceiling is 1857 tokens/s/GPU—the line every evaluation bar is measured against.

3.5 Where Existing Systems Lose

Against 1857 tokens/s/GPU the incumbents deliver only vLLM 22.0%, DeepSpeed-FastGen 22.9%, and TensorRT-LLM 37.8%—the best throws away nearly two-thirds of the machine. Figure 2 shows why: systems like SGLang [56], vLLM [16], and DeepSpeed-FastGen [12] run one batch through the layer executing operations *in sequence*, each a barrier for the next. During the memory-bound decode attention and the network AllReduce, the tensor cores sit idle—the “WASTED” bubbles in the figure—and aggregate compute utilization lands near 40%.

3.6 Intra-device Parallelism via Nano-batching

The sequential design loads each weight matrix once; splitting the batch reloads them, which seems to add expensive memory traffic. But not in a compute-bound regime: §3.3 showed memory bandwidth underutilized by more than half, so extra weight loads are charged against slack and, if overlapped with compute, cost zero wall-clock time. The system buys compute utilization with memory bandwidth it was not spending anyway.

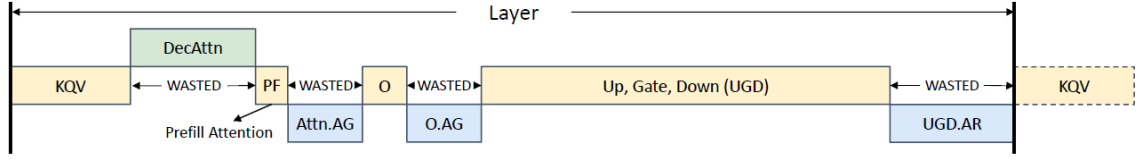


Fig. 2. Execution pipeline of existing systems, reproduced from the original paper. Green, yellow, and blue denote memory-, compute-, and network-bound operations respectively. “WASTED” marks the stages where compute—the constrained resource—is underutilized.

NanoFlow splits the batch: every operation becomes several nano-operations, bit-identical copies applied to disjoint *nano-batches*. An Up projection over 2048 tokens becomes UP1 (tokens 0–767) and UP2 (768–2047). Because the nano-batches share no data, UP2 depends neither on UP1 nor on the decode attention of nano-batch 1—the serializing barrier is gone, and a compute-bound and a memory-bound nano-operation from different nano-batches can run concurrently on disjoint hardware.

Two details matter. First, NanoFlow gives each nano-operation only a fraction R of the SMs: bandwidth-bound kernels saturate at few thread blocks while GEMMs absorb every SM, so an 80/20 split costs the GEMM almost nothing. Second, KV-cache offload (§3.8.2) is scheduled into compute-bound windows where PCIe traffic is invisible.

Algorithm 1 states the end-to-end procedure.

3.7 Auto-search

3.7.1 Profiling and the Interference Model. Auto-search needs two inputs.

Isolated kernel timings. NanoFlow profiles GEMM (dense), GEMV (decode attention), and network kernels at batch sizes $128 \dots B_{\text{Dense}}$ in steps of 128, sweeping block/warp/tile configurations and recording the fastest implementation per (kernel, batch size).

Interference. Harder: GPUs offer no clean way to hand each nano-operation a fixed fraction of compute, bandwidth, and network. Co-running kernels contend for SMs, warp slots, L2, and memory-controller queues [41], and the slowdown is not predictable from first principles. NanoFlow defines a *proxy*: it measures how much a co-running kernel degrades GEMM performance and calls that GEMM’s share R . If a co-scheduled GEMM runs at 40% of isolated speed, $R_A = 0.4$ and the partner gets $R_B = 1 - R_A = 0.6$; the definition is GEMM-centric because compute is what NanoFlow maximizes. Separately, P is each kernel’s achieved performance normalized to its isolated peak: $P_A = R_A$ for the GEMM by construction, while P_B says how much memory/network throughput the partner achieves. The mapping $R \mapsto P$ is thus an *exchange rate* between compute surrendered and memory/network gained.

Exhaustive profiling is infeasible, so NanoFlow prunes hard: GEMV and network kernels are limited to 8–128 thread blocks (128 saturates their bandwidth); dominated GEMM implementations are discarded; and only *pairwise* interference is profiled, assumed to hold under three-way overlap.

Table 3 distills the result, and its key feature is *convexity*: surrendering the first 20% of GEMM performance ($R = 1.0 \rightarrow 0.8$) buys 30% of GEMV or 50% of network performance, while the exchange rate turns unfavorable only at the far end. A scheduler that exploits only the cheap end of the curve captures most of the overlap benefit for little compute sacrifice—and Stage II’s MILP finds where to stop. The mapping is stable to within 5% across GEMM shapes and 64 batch combinations, so one table suffices.

Algorithm 1 NanoFlow: Auto-search and Runtime Execution

Require: Model architecture $\mathcal{M}$, hardware spec $\mathcal{H}$, workload statistics $\mathcal{W} = (p, d)$
Ensure: Optimized intra-device pipeline Π , served token stream

Phase 0: Profiling (offline, once per model+hardware)

- 1: $B_{\text{Dense}} \leftarrow \text{MAXDENSEBATCH}(\mathcal{M}, \mathcal{H}, \mathcal{W})$ ▷ largest batch fitting in memory
- 2: **for all** $k \in \{\text{GEMM}, \text{GEMV}, \text{Network}\}$ **do**
- 3: **for all** $b \in \{128, 256, \dots, B_{\text{Dense}}\}$ **do**
- 4: $D_{\text{best}}[k, b] \leftarrow \min_{\text{impl}} \text{PROFILEISOLATED}(k, b, \text{impl})$
- 5: **end for**
- 6: **end for**
- 7: $\mathcal{T}_{R \rightarrow P} \leftarrow \text{PROFILEPAIRWISEINTERFERENCE}()$ ▷ build Table 3

Phase 1: Pipeline structure search (MILP, no interference)

- 8: $n \leftarrow 2$ ▷ start with 2 nano-ops per operation
- 9: $\Pi \leftarrow \emptyset$
- 10: **repeat**
- 11: $\Pi' \leftarrow \text{SOLVEMILP}(n, B_{\text{Dense}}, \mathcal{G}_{\text{dep}}, D_{\text{best}})$
- 12: **minimize** makespan *s.t.* dependency, batch-size, and disjoint-resource-overlap constraints
- 13: **if** $\text{MAKESPAN}(\Pi') < \text{MAKESPAN}(\Pi)$ **then**
- 14: $\Pi \leftarrow \Pi'$
- 15: **end if**
- 16: $n \leftarrow n + 1$ ▷ split further near remaining bubbles
- 17: **until** no improvement in $\text{MAKESPAN}(\Pi)$

Phase 2: Resource allocation refinement (MILP, with interference)

- 18: Fix counts, sizes, and ordering from Π
- 19: $\{R_i\} \leftarrow \text{SOLVEMILP}(\Pi, \mathcal{T}_{R \rightarrow P})$
- 20: **minimize** makespan *s.t.* $\sum_{i \in \text{concurrent}(t)} R_i \leq 1.0 \forall t$,
 exec time of nano-op $i = D_{\text{best}}[i]/P(R_i)$
- 21: $\Pi \leftarrow \text{ANNOTATE}(\Pi, \{R_i\})$

Phase 3: Runtime serving (online)

- 22: **while** requests arrive **do**
- 23: **asynchronously** form batch for iteration $i+1$ *while* GPU runs iteration i
- 24: Prioritize unfinished decodes; chunk prefills to fill B_{Dense} exactly
- 25: Predict peak memory; admit new prefills only if within budget
- 26: **for all** layer $\ell = 1 \dots L$ **do**
- 27: **for all** nano-op $o \in \Pi$ **in order do**
- 28: Launch kernel impl. for (o, R_o) ; enforce deps via CUDA events
- 29: Offload KV of ℓ to CPU/SSD during FFN compute window
- 30: **end for**
- 31: **end for**
- 32: Detect EOS from iteration $i-1$; retire finished requests
- 33: **end while**
- 34: **return** served token stream

3.7.2 *Stage I: Pipeline Structure.* Stage I solves the pipeline *shape*—counts, sizes, ordering—via MILP over interference-free timings; interference is excluded to keep the problem tractable and corrected in Stage II. The formulation:

Objective. Minimize makespan—eliminate the compute bubbles of Figure 2.

Table 3. Performance P of GEMV and network kernels at different resource utilization levels R . Recreated from Table 3 of the original paper. Note the convexity: the first 20% of GEMM performance surrendered buys 30% GEMV or 50% network performance.

Operation	Resource Utilization (R)						
	0	0.1	0.2	...	0.8	0.9	1
GEMM (by defn.)	0	0.1	0.2	...	0.80	0.90	1
GEMV	0	0.2	0.3	...	0.85	0.95	1
Network	0	0.3	0.5	...	0.90	1.00	1

Nano-operation count. At least 2 per operation (one cannot overlap with itself), but splitting erodes batching, so auto-search starts at 2, solves, and selectively increases the split only for operations adjacent to a remaining bubble, stopping when it stops helping.

Batch sizes and timings. Sizes from $\{128, \dots, B_{\text{Dense}}\}$; timings from the isolated profile.

Dependencies. Two nano-operations are dependent iff their parents are dependent *and* their token ranges intersect—so O-projection over tokens 0–255 depends on attention over 0–255 but not 256–511. This conjunctive rule generates the parallelism.

Overlap. Only operations of *different* resource classes may overlap; two GEMMs sharing tensor cores gain nothing.

Operation transformations. Algebraically equivalent collectives (an AG can become an AR under a different sharding [37, 55]) are enumerated, hence the “AG to AR Transform” in Figure 3. Rather than prove optimality, NanoFlow accepts the first good feasible solution (≈ 10 minutes).

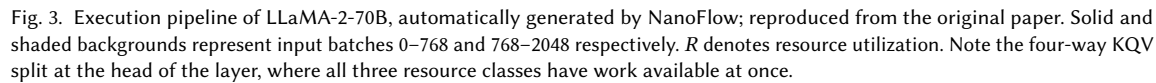
3.7.3 Stage II: Resource Allocation. Stage II freezes Stage I’s structure and solves a second MILP for the resource shares $\{R_i\}$, using Table 3 to map each R_i to an achieved P_i (and a concrete kernel implementation). Constraints: $\sum_i R_i \leq 1.0$ at any instant; nano-operation i takes $D_{\text{best}}[i]/P(R_i)$; objective, minimize makespan. Auto-search reruns only when the architecture or workload statistics change materially—negligible against week-long deployments.

3.7.4 Discovered Pipelines. 70B-class models (LLaMA-2/3-70B, Qwen2.5-72B, Deepseek-67B) differ in vocabulary, layer count, and KQV bias, but none perturbs the resource profile enough to change the schedule; auto-search converges to near-identical pipelines for all four. Figure 3 shows the LLaMA-2-70B result: at the layer head all three resource classes have work (KQV compute, decode-attention memory, the previous layer’s AllReduce), so KQV is split *four* ways; decode attention takes $R = 0.4$ and achieves 80% of its peak—the convex bargain of Table 3—while the compute-dominated FFN prioritizes GEMMs ($R = 0.9$) with two nano-operations.

8B models (LLaMA-3-8B) fit on one GPU—no network operations—so everything splits in two, overlapping decode attention against Up-Gate-Down. **MoE models** (Mixtral 8×7B) have imbalanced expert load [15], so NanoFlow uses tensor parallelism with a grouped-GEMM FFN and a gate-routing step; auto-search handles the modified graph without special-casing.

3.8 The Runtime

3.8.1 Request Scheduling. Batch formation. NanoFlow assumes autoscaling and routing are handled by an external control plane [14, 32, 44] and that requests are plentiful. Following Sarathi-Serve [3], it prioritizes unfinished decodes, then chunks prefills at token granularity to hit a fixed best-performing B_{Dense} every iteration. Constant batch shape stabilizes GEMM performance (p99 latency within $1.07\times$ of mean) and self-balances—as decodes accumulate the prefill



Asynchronous scheduling. CPU-side batch formation (memory projection, admission, PagedAttention [16] page-table updates) is not free [40]. Conventional systems [16, 56] do it *after* the GPU finishes an iteration, idling the GPU; NanoFlow instead builds iteration $i+1$ while iteration i runs. The cost is one-iteration EOS blindness—one wasted token per request, under 1% given mean decode lengths above 100 (Table 4)—in exchange for fully hiding the scheduler.

Simultaneous offloading. NanoFlow offloads each layer’s KV tensors *immediately* after KQV generation, while they are still contiguous and cheap to copy. Constant dense batch size makes per-iteration offload volume constant, so PCIe traffic is smooth; the GPU-initiated copy is scheduled into the FFN’s compute-bound window, and NUMA-aware thread binding trims host cost.

Loading and scattering. Since PagedAttention scatters KV pages across GPU memory, NanoFlow first DMAs host data into a contiguous staging buffer and scatters on-device—worth 7–10× on host-to-device bandwidth.

NanoFlow is roughly 10K lines of CUDA and 6K of Python for NVIDIA GPUs; auto-search’s output selects the profiled kernel implementation realizing each nano-operation’s R , and ordering is enforced with CUDA events.

4.1 Setup

Models. LLaMA-2-70B primarily, plus LLaMA-3-70B, LLaMA-3-8B, Qwen2-72B, Deepseek-67B, and Mixtral 8×7B, all FP16.

Table 4. The average and standard deviation of input and output lengths in the sampled datasets. Recreated from Table 4 of the original paper. The three traces span a wide range of prefill/decode ratios, which is why NanoFlow’s advantage is not a workload artifact.

Dataset	Avg. Input (Std)	Avg. Output (Std)
Splitwise [31]	1155 (1109)	211 (163)
LMSYS-Chat [54]	102 (169)	222 (210)
ShareGPT [1]	246 (547)	322 (244)

Baselines. vLLM [16, 50] (v0.5.3), DeepSpeed-FastGen [12, 22] (v0.2.3), and TensorRT-LLM [25, 26] (v0.8.0), each tuned for peak throughput with paged KV-cache and dynamic/chunked batching.

Datasets. Three traces (Table 4) spanning a wide range of prefill/decode ratios: the production Splitwise [31] trace (long prompts), LMSYS-Chat-1M [54] (short, decode-heavy), and ShareGPT [1] in between; 50K requests sampled from the latter two, the full Splitwise trace.

4.2 Offline Throughput

NanoFlow runs at $B_{\text{Dense}} = 2048$ against a workload-independent optimum of 1857 tokens/s/GPU (Eq. 6). It wins every configuration, peaking at 68.5% of the ceiling. On constant-length workloads it averages 2.62× vLLM, 2.78× DeepSpeed-FastGen, and 1.73× TensorRT-LLM; on dataset-derived lengths the margins widen to 4.18×, 3.45×, and 1.91×. The widening is diagnostic: variable real-trace lengths make the baselines’ batch shapes fluctuate, degrading GEMM efficiency, whereas NanoFlow’s constant- B_{Dense} policy always assembles the same dense shape. The gap grows precisely where production lives.

4.3 Latency

Throughput at unusable latency would be a poor trade, so the authors model Poisson arrivals [16], sweep request rate, and measure *normalized latency* (end-to-end latency per output token) against a 200 ms/token SLO [8, 57]. At low load NanoFlow is *slightly worse*—a system that always assembles a 2048-token batch makes a lone request wait—but as load rises the baselines’ latency turns sharply upward while NanoFlow’s stays flat, sustaining 1.64× TensorRT-LLM’s request rate on LMSYS-Chat-1M within the SLO. Tail latency is controlled: p99 is only 1.07× the mean at near-peak throughput, a consequence of the constant batch size.

4.4 Ablation

To isolate contributions, the authors build two same-runtime baselines: *non-overlap* (one batch, sequential ops) and *nanobatch-only* (split but still sequential). The honest first number: *nano-batching alone costs 13.2%*—the extra weight loads are real, and NanoFlow must repay this tax before it gains anything. It does: overlapping network with compute recovers 1.07×, and overlapping both network and memory with compute recovers 1.17× over the non-overlapped baseline. KV-cache offloading costs 3.0% but eliminates 3.02× of redundant prefill compute on multi-round LMSYS-Chat workloads—no contest for conversational deployments.

4.5 Resource Utilization

The most direct evidence traces compute, memory, and network utilization through a single layer. The non-overlapping baseline’s traces are nearly disjoint—only one resource is ever busy—whereas NanoFlow’s are simultaneous: compute stays high across the whole layer while memory and network ride underneath, lifting average compute utilization to 68.5% against roughly 40%. It is not 100%: kernel interference (Table 3) means a GEMM sharing the device with a GEMV cannot run at its isolated peak, and the exchange rate’s convexity bounds how far the gap closes.

4.6 Generalization Across Models

Finally, whether auto-search generalizes: five additional models at constant 1024-in/512-out, all on 8×A100 except single-GPU LLaMA-3-8B. NanoFlow reaches 50–72% of optimal on every one, averaging a 2.66× gain over vLLM. Two extremes are telling: LLaMA-3-8B hits 78.5% (the best result, and the only single-GPU case with no network operations to schedule around), while Mixtral 8×7B reaches only 50.4%, capped by expert-routing load imbalance [15] that lives in the data, not the pipeline.

5 Discussion

5.1 What the Paper Actually Establishes

Stripped down, NanoFlow makes one conceptual and one engineering claim. The *conceptual* claim is that the memory-bound framing of LLM serving is a category error—true of an operation, false of a system. It survives scrutiny because the authors derive it (Eq. 5), measure it against real kernels (Table 2: 114 ms compute vs. 45 ms memory), and show it is not an accident of one GPU (Table 1). It is also the more valuable contribution: if the bottleneck is memory, quantization and KV compression are the frontier; if it is compute, those buy slack already available and the frontier is *scheduling*. The *engineering* claim—that intra-device parallelism converts that slack into throughput—is honestly tested by the ablation, where nano-batching alone *loses* 13.2% and only overlapping repays it. That the authors lead with this number is to their credit.

5.2 Assessment

Strengths. The cost model is clean: Eq. 6 reduces the throughput ceiling to $\text{Compute}/2P_{\text{Model}}$, evaluable on a napkin for any (model, hardware) pair and reusable beyond NanoFlow. The two-stage auto-search rightly separates the combinatorial structure (counts, sizes, ordering) from the continuous one (resource shares). And the empirical posture is consistently self-critical—prefill-attention misprediction, low-load latency regression, and the 13.2% nano-batching cost are all disclosed.

Limitations. *The optimum is not reached:* 68.5% leaves 31.5% of the machine idle, attributed to kernel interference whose convexity (Table 3) suggests a hard ceiling that may need hardware support for resource partitioning. *Low-load latency regresses*, disclosed but unsolved—the implicit fix (run fewer NanoFlow instances) pushes the problem into a control plane the paper declines to build. *The interference model is doubly approximate:* pairwise profiles are assumed to compose to three-way overlaps, and the $R \mapsto P$ mapping is an empirical fit validated to 5%. *The compute-bound conclusion is contingent:* T_R depends on $\text{Compute}/\text{MemBW}$, and aggressive weight quantization or long-context serving can flip it—the paper’s own $T_R \approx 1$ for LLaMA-3-8B at 512-in/1024-out previews the boundary. *MoE is a weak spot:* 50.4% is the worst result, and expert-load imbalance is data-dependent, arguably the most consequential gap given MoE’s rise.

5.3 Broader Implications

Two lessons generalize. First, *measure the system, not the operation*: the memory-bound consensus rested on good measurements of the wrong unit of analysis, and per-operation bottleneck reasoning can be individually correct yet collectively misleading in any heterogeneous pipeline. Second, *heterogeneity is an opportunity*: rather than eliminating a memory-bound kernel in a compute-bound pipeline, NanoFlow schedules it as a free rider consuming an otherwise-idle resource—applicable to any system mixing resource classes, from recommendation embeddings to sparse/dense scientific pipelines.

5.4 Future Directions

An n -way interference model (rather than composed pairwise profiles) could pack more aggressively and recover some of the missing 31.5%. Long-context regimes eventually push T_R above 1, expiring NanoFlow’s premise, so detecting and adapting to that crossover is the obvious next problem. Aggressive INT4/FP8 quantization moves both terms of T_R and may force re-derivation of the pipeline. Extending overlap *across* devices—one GPU’s network phase against another’s compute—appears tractable. And MoE-aware nano-batching that respects expert routing could rebalance the load currently capping Mixtral at 50.4%.

5.5 Reproduction Notes

This report reproduces the paper’s argument, structure, and reported results from the published manuscript; it does not re-run the experiments, which need an 8×A100 node. Figures are reproduced from the original and tables recreated from reported values. The key derivations were re-checked: substituting Compute = 280 TFLOPS and $P_{\text{Model}} = 70\text{B}$ into Eq. 6 reproduces 1857 tokens/s/GPU, and $P_{\text{Model}} \approx 12D_{\text{model}}^2 L$ follows from the stated intermediate-dimension ratio.

6 Conclusion

NanoFlow overturns the assumption that LLM serving is memory-bound, showing analytically and empirically that batched serving of modern models is compute-bound: the individual operations remain memory-bound, but the system built from them is not. Existing engines execute compute-, memory-, and network-bound operations sequentially, idling the compute units for most of every layer and reaching only about 40% utilization. NanoFlow’s answer is intra-device parallelism—splitting batches into nano-batches and replicating operations into independent nano-operations to dissolve the serializing dependencies, then using a two-stage MILP auto-search to set the number, ordering, and GPU share of each nano-operation under measured interference. The result lifts compute utilization to 68.5% and delivers 1.91× the throughput of the best prior system on realistic traces, holding p99 latency within 1.07× of the mean and generalizing without tuning to six architectures.

The system does not reach its derived ceiling and does not claim to: interference costs roughly a third of the machine, low-load latency regresses, and MoE imbalance is unaddressed. Its durable contribution is the principle that in a pipeline of heterogeneous operations, the resource one kernel leaves idle is free for another to use—and that identifying which resource binds at the level of the *system* rather than the *operation* is the prerequisite for knowing where to optimize.

References

- [1] 2023. ShareGPT. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. Accessed: 2025-01-15.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. *arXiv preprint arXiv:2403.02310* (2024).

Manuscript submitted to ACM

- [3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [4] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 4895–4901.
- [5] Alibaba Cloud. 2024. Alibaba Cloud’s Qwen2 with Enhanced Capabilities Tops LLM Leaderboard. <https://www.alibabacloud.com/blog/>.
- [6] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. *arXiv:1607.06450*
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.
- [8] Marc Brysbaert. 2019. How Many Words Do We Read Per Minute? A Review and Meta-Analysis of Reading Rate. *Journal of Memory and Language* 109 (2019), 104047.
- [9] Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2017. Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning. *arXiv:1702.03118*
- [10] Erin Griffith. 2023. *The Desperate Hunt for the A.I. Boom’s Most Indispensable Prize*. Technical Report. The New York Times.
- [11] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. FasterMoE: Modeling and Optimizing Training of Large-Scale Dynamic Pre-Trained Models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP ’22)*. Association for Computing Machinery, New York, NY, USA, 120–134.
- [12] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, et al. 2024. DeepSpeed-FastGen: High-Throughput Text Generation for LLMs via MII and DeepSpeed-Inference. *arXiv preprint arXiv:2401.08671* (2024).
- [13] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. *arXiv:1811.06965*
- [14] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. 2025. Intelligent Router for LLM Workloads: Improving Performance through Workload-Aware Load Balancing.
- [15] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv:2310.06825*
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [17] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.
- [18] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. *arXiv:2405.04532*
- [19] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 881–897.
- [20] Yusuf Mehdi. 2023. Reinventing Search with a New AI-Powered Microsoft Bing and Edge, Your Copilot for the Web.
- [21] Meta. 2024. Build the Future of AI with Meta Llama 3. <https://llama.meta.com/llama3/>.
- [22] Microsoft. 2024. DeepSpeed: FastGen. <https://github.com/microsoft/DeepSpeed/tree/429bc5c/blogs/deepspeed-fastgen>. Commit ID: 429bc5c, Accessed: 2024-12-09.
- [23] Mistral AI. 2024. Mistral NeMo. <https://mistral.ai/news/mistral-nemo/>.
- [24] NVIDIA. 2024. NVLink and NVLink Switch. <https://www.nvidia.com/en-us/data-center/nvlink/>.
- [25] NVIDIA. 2024. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>.
- [26] NVIDIA. 2024. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM/tree/5955b8a>. Commit ID: 5955b8a, Accessed: 2024-12-09.
- [27] OpenAI. 2023. Introducing ChatGPT.
- [28] OpenAI. 2024. GPT-4 Technical Report. *arXiv:2303.08774*
- [29] Jongseok Park, Kyungmin Bin, Gibum Park, Sangtae Ha, and Kyunghan Lee. 2023. ASPEN: Breaking Operator Barriers for Efficient Parallelization of Deep Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 68625–68638.
- [30] Dylan Patel and Afzal Ahmad. 2023. The Inference Cost of Search Disruption – Large Language Model Cost Analysis.
- [31] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. *arXiv:2311.18677*
- [32] Archit Patke, Dharmath Reddy, Saurabh Jha, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2025. Hierarchical Autoscaling for Large Language Model Serving with Chiron.

- [33] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. *Proceedings of Machine Learning and Systems* 5 (2023).
- [34] Emma Roth. 2024. ChatGPT's Weekly Users Have Doubled in Less Than a Year. Accessed: 2024-12-03.
- [35] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling Deep Learning Memory Access via Tile-Graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 701–718.
- [36] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053
- [37] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053. arXiv:1909.08053v4
- [38] Shubham Singh. 2024. ChatGPT Statistics (AUG 2024) – Users Growth Data. <https://www.demandsage.com/chatgpt-statistics/>. Accessed: 14-08-2024.
- [39] Jared Spataro. 2023. Introducing Microsoft 365 Copilot – Your Copilot for Work.
- [40] Vikranth Srivatsa, Dongming Li, Yiyang Zhang, and Reyna Abhyankar. 2024. Can Scheduling Overhead Dominate LLM Inference Performance? A Study of CPU Scheduling Overhead on Two Popular LLM Inference Systems. https://mlsys.wuklab.io/posts/scheduling_overhead/. Accessed: 25-10-2024.
- [41] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-Aware, Fine-Grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 1075–1092.
- [42] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. arXiv:2406.10774
- [43] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, et al. 2023. CUTLASS.
- [44] The AIBrix Team, Jiaxin Shan, Varun Gupta, Le Xu, Haiyang Shi, Jingyuan Zhang, Ning Wang, Linhui Xu, Rong Kang, Tongping Liu, et al. 2025. AIBrix: Towards Scalable, Cost-Effective Large Language Model Inference Infrastructure.
- [45] Cheng Ting-Fang. 2023. *TSMC Sees AI Chip Output Constraints Lasting 1.5 Years*. Technical Report. Nikkei Asia.
- [46] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971
- [47] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. LLaMA 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).
- [48] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Pat McCormick, Jamaludin Mohd-Yusof, et al. 2022. Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 267–284.
- [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [50] vLLM Project. 2024. vLLM. <https://github.com/vllm-project/vllm/tree/38c4b7e>. Commit ID: 38c4b7e, Accessed: 2024-12-09.
- [51] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [52] Liangyu Zhao, Saeed Maleki, Aashaka Shah, Ziyue Yang, Hossein Pourreza, and Arvind Krishnamurthy. 2024. ForestColl: Efficient Collective Communications on Heterogeneous Network Fabrics. arXiv:2402.06787
- [53] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2023. Atom: Low-Bit Quantization for Efficient and Accurate LLM Serving. *arXiv preprint arXiv:2310.19102* (2023).
- [54] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhenhao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2023. LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset. arXiv:2309.11998
- [55] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yanzhong Xu, Danyang Zhuo, Eric P. Xing, et al. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
- [56] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104
- [57] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. *arXiv preprint arXiv:2401.09670* (2024).